

Softwarekonstruktion – Übung 1

Organisatorisches

Präsenzübungen

- Kurzvorstellung wesentlicher Fallstricke aus den Hausübungen (ca. 5-10 min)
- Bearbeitung der Aufgaben in Gruppen (ca. 60 min)
- Gemeinsames Besprechen der Aufgaben (ca. 20-25 min)
- Die Tutoren stehen für Fragen bei der Bearbeitung zur Verfügung, es ist aber **keine** Vorrechenübung

Hausübungen

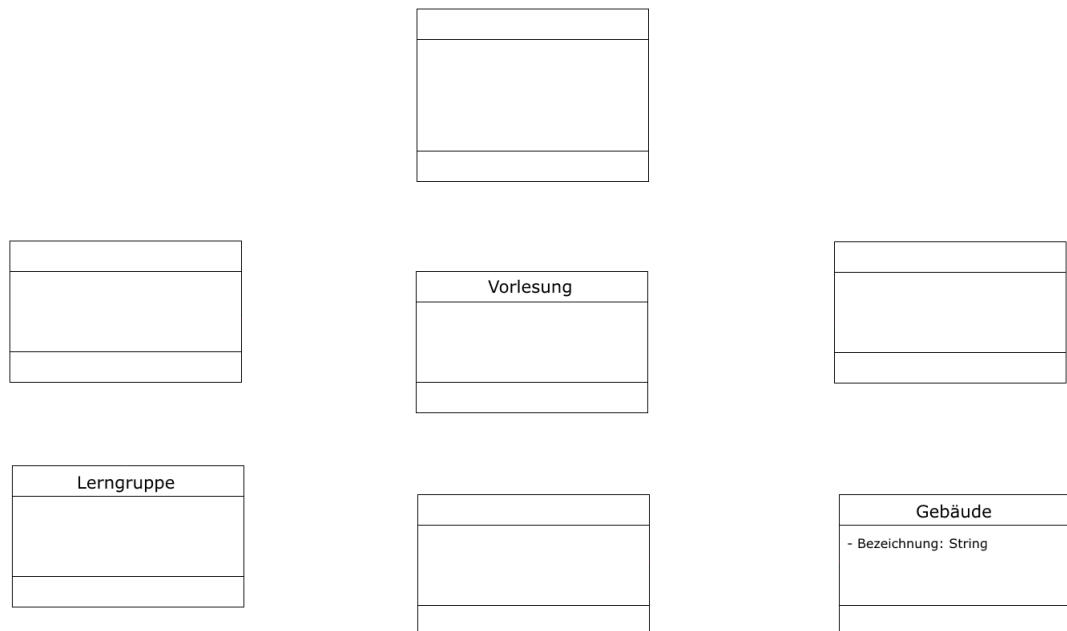
- Übungezettel:
 - Es wird insgesamt 6 Hausübungen mit insgesamt 100 Punkten geben.
 - Bei den letzten beiden Hausübungen handelt es sich voraussichtlich um Online-Übungen.
 - Die Abgabe erfolgt in Gruppen von 2-4 Teilnehmern.
 - Deadlines der Abgaben sowie Informationen zur Online-Übung sind der Webseite der Veranstaltung zu entnehmen.
- Mögliche Abgabe:
 - in den Übungen
 - Einwurf in die Briefkästen Nr. 49-51 (je nach Gruppe) in der Otto-Hahn-Str. 12
- Keine Abgabe:
 - per Mail oder Hauspost
 - persönlich am Lehrstuhl oder Sekretariat
- Rückgabe:
 - in der entsprechenden Übung
 - nicht abgeholte Übungszeitel liegen im Sekretariat zur Abholung bereit
- Zum Erbringen der Übungsleistung:
 - Insgesamt sind mind. **50% der Gesamtpunkte** zu erreichen, dabei aber mind. **30% in der 1. Hälfte der Übungszeitel** und **30% in der 2. Hälfte der Übungszeitel** (d.h. mind. 50 Punkte in Summe, davon mind. 15 Punkte aus den Zeiteln 1-3 und mind. 15 Punkte aus den Zeiteln 4-6)
- **Abgabe von Duplikaten führt zu 0 Punkten für alle beteiligten Gruppen**

1 Object Constraint Language (OCL)

1.1 UML-Klassendiagramm

Ergänzen Sie das unten stehende UML-Klassendiagramm gemäß nachfolgender Beschreibung. Vervollständigen Sie dabei die Klassen *Person*, *Student*, *Professor*, *Vorlesung*, *Lerngruppe*, *Hörsaal* und *Gebäude* und verbinden diese entsprechend.

Verwenden Sie dabei Aggregationen, Kompositionen, Multiplizitäten und Leserichtungen. Nutzen Sie auch das Konzept der Vererbung. Geben Sie zudem zu allen Attributen passende Datentypen an.



1. Personen besitzen einen Vornamen, einen Nachnamen und ein Alter.
2. Studenten sind Personen, die außerdem eine Matrikelnummer besitzen.
3. Professoren sind ebenfalls Personen, allerdings besitzen sie aufgrund ihrer Anstellung eine Personalnummer und beziehen Gehalt, eine Matrikelnummer besitzen sie natürlich nicht.
4. Studenten besuchen Vorlesungen, Professoren halten Vorlesungen. Professoren sind dabei Lehrende, Studenten die Lernenden. Studenten können einsehen, welche Vorlesungen sie besucht haben, Vorlesungen hingegen haben keinen Zugriff auf die Listen ihrer Besucher.
5. Vorlesungen finden in Hörsälen statt. Jeder Hörsaal besitzt eine Nummer.
6. Ein Hörsaal gehört zu einem Gebäude.
7. Studenten können Lerngruppen bilden.

1.2 OCL Begriffe

Erläutern Sie stichpunktartig nachfolgende Begriffe im Zusammenhang mit der OCL. Beschreiben Sie dabei die Bedeutung der Begriffe und geben Sie entsprechende OCL-Schlüsselwörter an.

1. Klasseninvariante
2. Vorbedingung bzw. Nachbedingung

In der Vorlesung sowie der Übung wird OCL in der Version 2.4 behandelt. Die Spezifikation ist bei Bedarf unter <http://www.omg.org/spec/OCL/2.4/PDF/> zu finden.

1.3 Vordefinierte Operationen

Die OCL ist eine sehr mächtige Sprache, daher wird in der Veranstaltung eine dedizierte Teilmenge der in OCL vordefinierten Operationen betrachtet. Machen Sie sich mit den nachfolgenden Operationen vertraut, indem Sie stichpunktartig beschreiben, was die jew. Operationen berechnen.

Wenn ein Klassendiagramm gegeben ist, so wenden Sie die entsprechende Operation als gültige Invariante auf alle Klassen an!

• logische Operatoren

- and, or, xor Konjunktion (und), Disjunktion (oder) sowie Kontravalenz
- not, implies _____
- drücken Sie implies nur mit Hilfe der anderen logischen Operatoren aus: _____

• Aufzählungstypen (enumeration types)

- *variable = enumeration name :: enumeration value* Enumerations sind Datentypen der UML, welche die Definition einer Reihe von Literalen als mögliche Werte erlauben. Für den Zugriff auf die Werte ist die :: Syntax zu verwenden.

• Operationen auf Objekten

1. boolesche Operatoren (wenn *self* mit Objekt *o* vergleichbar ist)

- *=(o:T): Boolean* gibt true zurück, wenn *self* gleich dem Objekt *o* ist
- *<>(o:T): Boolean* gibt true zurück, wenn *self* ungleich dem Objekt *o* ist
- *<(o:T): Boolean* gibt true zurück, wenn *self* kleiner als *o* ist
- *<=(o:T): Boolean* gibt true zurück, wenn *self* kleiner gleich *o* ist
- *>(o:T): Boolean* gibt true zurück, wenn *self* größer als *o* ist
- *>=(o:T): Boolean* gibt true zurück, wenn *self* größer gleich *o* ist

- **Operationen auf Collections**

1. **einfache Operationen**

- $\rightarrow \text{size}(): \text{Integer}$ _____
- $\rightarrow \text{sum}(): \text{Real}$ _____
- $\rightarrow \text{isEmpty}(): \text{Boolean}$ _____
- $\rightarrow \text{notEmpty}(): \text{Boolean}$ gibt true zurück, wenn *Collection* mindestens ein Element enthält

2. **Elementbezogene Operationen**

- $\rightarrow \text{includes}(o:T): \text{Boolean}$ _____
- $\rightarrow \text{excludes}(o:T): \text{Boolean}$ gibt true zurück, wenn *o* nicht in der *Collection* enthalten ist
- $\rightarrow \text{count}(o:T): \text{Integer}$ _____
- $\rightarrow \text{isUnique}(expr:\text{OclExpression}): \text{Boolean}$ _____

3. **Iterator-Ausdrücke**

- $\rightarrow \text{select}(expr:\text{OclExpression}): \text{Collection}(T)$ _____
- $\rightarrow \text{collect}(expr:\text{OclExpression}): \text{Collection}(T)$ _____
- $\rightarrow \text{reject}(expr:\text{OclExpression}): \text{Collection}(T)$ liefert eine Collection, die alle Elemente von *self* enthält, außer diejenigen, welche die *expr* erfüllen
- $\rightarrow \text{forAll}(expr:\text{OclExpression}): \text{Boolean}$ _____

- **Operationen auf Klassen**

- $\text{classifier.allInstances}(): \text{Set}(T)$

Student

Es soll nicht mehr als 10.000 Studenten geben:

context Student

Das UML-Klassendiagramm in Abbildung 1 erlaubt leider noch die Konstruktion ungewollter Konstellationen. Daher muss die OCL verwendet werden, um notwendige Randbedingungen durch die Spezifikation von Einschränkungen (constraints) zu formulieren. Drücken Sie folgende Constraints in Form von gültigen OCL-Ausdrücken aus

**Formulieren Sie in dem Kontext, den die Aufgabenstellung impliziert.
Verwenden Sie dabei ausschließlich die Operationen aus Aufgabe 1.3.**

1. Das Alter eines Menschen ist immer positiv.
2. Eine Veranstaltung können maximal so viele Gäste besuchen, wie der entsprechende Veranstaltungsort fassen kann.
3. Die Veranstaltungshalle *Westfalahalle* muss für jede Veranstaltung mindestens 60 Karten für den freien Verkauf bereitstellen.
4. Aus wirtschaftlichen Gründen muss die *Westfalahalle* sparen. Daher dürfen die Gehälter aller Arbeitnehmer der Westfalahalle 60% des Budgets der Halle nicht übersteigen.
5. Bei einem Flohmarkt darf es in der Veranstaltungshalle keine Bestuhlung geben.

Hausübung

1.5 Mengen in OCL (5,5 Punkte)

1. Erläutern Sie stichpunktartig die Unterschiede zwischen **Bag**, **Set**, **OrderedSet** und **Sequence**!
2. Geben sie jeweils eine Lösung für die nachfolgenden OCL Ausdrücke an. Schlagen sie bei Bedarf die entsprechende Stelle in der OCL Spezifikation nach. Sollte ein Ausdruck keine gültige Anfrage sein, so notieren Sie dies.

Beispiel: $\text{Set}\{1,4,7,10\} - \text{Set}\{4,7\} = \text{Set}\{1,10\}$

- $\text{Set}\{1,2,3,4,5\} \rightarrow \text{sum}() =$
- $\text{Set}\{1,2,3,4,5\} \rightarrow \text{size}() =$
- $\text{Sequence}\{ 'h', 'a', 'l', 'l', 'o' \} \rightarrow \text{last}() =$
- $\text{Bag}\{ 'h', 'a', 'l', 'l', 'o' \} \rightarrow \text{last}() =$
- $\text{Set}\{ 'h', 'a', 'l', 'l', 'o', ' ', 'w', 'e', 'l', 't' \} \rightarrow \text{one}(e|e='a') =$
- $\text{Set}\{ 'h', 'a', 'l', 'l', 'o', ' ', 'w', 'e', 'l', 't' \} \rightarrow \text{select}(e|e='l') \rightarrow \text{size}() =$
- $\text{Set}\{1,2,-1,-2\} \rightarrow \text{isUnique}(e|e*e) =$

1.6 OCL Anwendung II (8,5 Punkte)

Betrachten Sie das UML-Klassendiagramm zu Veranstaltungshallen aus Aufgabe 1.4. Drücken Sie folgende Constraints in Form von korrekten OCL-Ausdrücken aus!

Verwenden Sie dabei ausschließlich die Operationen aus Aufgabe 1.3!

1. Manager verdienen mehr als 5.000 Euro, Angestellte und Küchenpersonal jedoch maximal 2500 Euro.
2. Jedes Ticket besitzt eine eindeutige ID.
3. Es darf nur bei Flohmärkten Verkäufer geben, bei allen anderen Veranstaltungen sind alle Gäste Besucher! Zudem muss es bei Flohmärkten mindestens einen Verkäufer geben.
4. Bei Flohmärkten dürfen pro Verkäufer in der *Westfalenhalle* nicht mehr als 5 Verkaufsstände aufgestellt werden.
5. Eine Theateraufführung muss mehr als 10 Besucher haben (um nicht abgesagt zu werden)! Zudem darf dann in der Veranstaltungshalle die Bestuhlung nicht fehlen.
6. Kultur soll bereits in der Jugend gefördert werden. Aus diesem Grund müssen Kinder unter 6 Jahren keinen Eintritt zu Klassikkonzerten oder Musicals zahlen, d.h. Tickets dieser Veranstaltungskategorien sind für sie kostenlos.

1.7 Vordefinierte Operationen - Teil 2 (6 Punkte)

Machen Sie sich mit den nachfolgenden Operationen vertraut, indem Sie stichpunktartig beschreiben, was die Operationen berechnen! Wenn ein Klassendiagramm gegeben ist, so wenden Sie die entsprechende Operation als gültige Invariante auf alle Klassen an, d.h. formulieren Sie mittels der Operation alle gültigen Invarianten zu dem Klassendiagramm!

• weitere Operationen auf Collections

1. boolesche Operatoren

- $=$ (c: Collection(T)): Boolean _____
- $<>$ (c:Collection(T)): Boolean _____

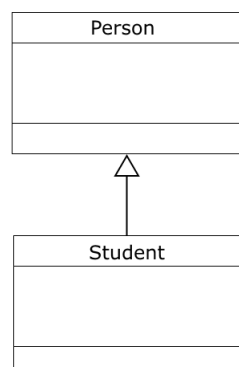
2. Mengen-Operationen

- $\text{set} \rightarrow \text{union}(c:\text{Collection}(T)):\text{Collection}(T)$ _____
- $\text{bag} \rightarrow \text{union}(c:\text{Collection}(T)):\text{Bag}(T)$ Die Vereinigung einer ungeordneten Menge mit Duplikaten (bag) mit einer Collection liefert wieder eine ungeordnete Menge, die Duplikate enthalten kann!
- $\text{set} \rightarrow \text{intersection}(c:\text{Collection}(T)):\text{Set}(T)$ _____
- $\text{bag} \rightarrow \text{intersection}(c:\text{Collection}(T)):\text{Collection}(T)$ Der Schnitt einer ungeordneten Menge mit Duplikaten (bag) mit einer Collection liefert eine Menge, welche den Typen der Collection hat!
- $\rightarrow \text{includesAll}(c:\text{Collection}(T))$: Boolean _____

• weitere Operationen auf Objekten

1. Typ vs. Art

- $\text{self.oclIsTypeOf}(\text{typespec})$: Boolean _____
- $\text{self.oclIsKindOf}(\text{typespec})$: Boolean _____



context Person

$\text{inv} : \text{self.oclIsTypeOf}(\text{Person})$

context Student

$\text{inv} : \text{self.oclIsTypeOf}(\text{Student})$

2. **undefiniert vs. ungültig**

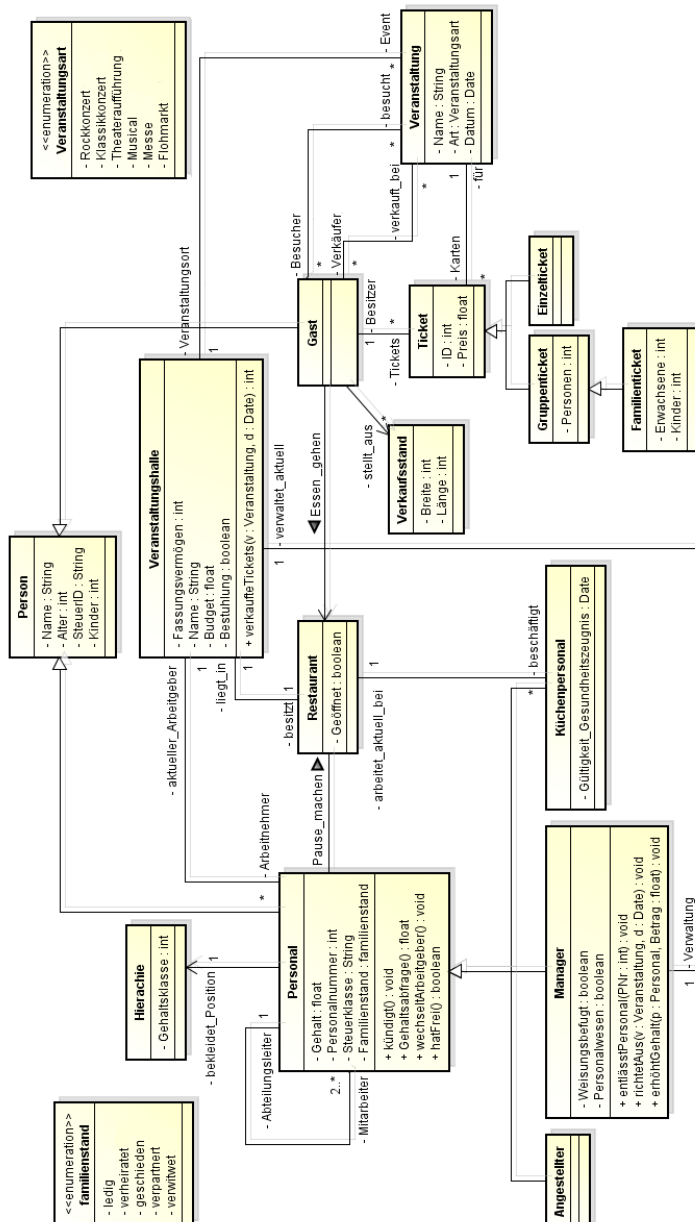
- self.ocIsUndefined(): Boolean _____

- self.ocIsInvalid(): Boolean _____

Softwarekonstruktion – Übung 2

2 OCL

2.1 OCL Anwendung III



Betrachten Sie das von Übungszettel 1 bekannte UML-Klassendiagramm in Abb.1.

Drücken Sie folgende Constraints in Form von OCL-Ausdrücken aus.

Verwenden Sie dabei ausschließlich die Operationen, die auf Übungsblatt 1 (in den Aufgaben 1.3 und 1.7) eingeführt wurden.

1. Die Anzahl von Personen eines Gruppentickets darf nicht negativ oder 0 sein!
2. Für Tickets gelten besondere Bestimmungen: Gruppentickets werden erst ab 2 Personen ausgestellt, bei Familientickets müssen mindestens 2 Erwachsene und 1 Kind gebucht werden!
3. Das Personal kann sein Gehalt über Gehaltsabfrage() abfragen. Dabei wird das aktuelle Gehalt zurück gegeben!
4. Wird das Gehalt des Personals um einen Betrag erhöht, so ist das neue Gehalt danach um genau diesen Betrag größer als das Gehalt zuvor. Damit die Gehaltserhöhung überhaupt stattfindet, muss der Betrag, um welchen das Gehalt erhöht werden soll, mindestens 3 % des aktuellen Gehalts betragen!
5. Bei dem Personal gelten die Bestimmungen, dass das Küchenpersonal über ein aktuell gültiges Gesundheitszeugnis verfügen muss!
6. Kündigt Jemand vom Personal einer Veranstaltungshalle, so gehört er anschließend nicht mehr zu den Arbeitnehmern der Veranstaltungshalle. Damit derjenige überhaupt kündigen kann, muss er natürlich vorher bei dieser Veranstaltungshalle angestellt gewesen sein!

EPK

2.2 Grundlagen

2.2.1 Mit welchen EPK-Elementen würden Sie folgende Aussagen modellieren?

1. Router konfigurieren
2. Werbebroschüre im Briefkasten
3. DSL-Anschluss ist geschaltet
4. Störungsstelle anrufen
5. DSL-Hardware versenden

2.2.2 Was versteht man bei EPKs unter einem Konnektor? Welche Arten gibt es?
 Erklären Sie diese in eigenen Worten.

2.3 Konnektoren

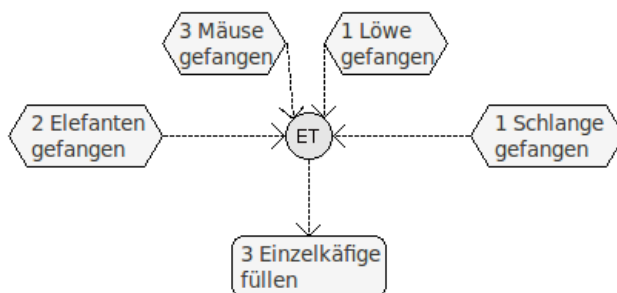
2.3.1 Beim Einsatz von Konnektoren unterscheidet man nach Ereignis- und Funktionsverknüpfung, sowie auslösende und erzeugte Ereignisse. Was könnte man darunter verstehen? Modellieren Sie jeweils ein Beispiel.

	Ereignisverknüpfung	Funktionsverknüpfung
auslösende Ereignisse		
erzeugte Ereignisse		

- 2.3.2 Eine der Variante aus Aufgabe 2.3.1 ist nur für den AND-Konnektor zulässig. Welche ist das und warum?
- 2.3.3 Funktionieren die in der Vorlesung vorgestellten Konnektoren auch für das zusammenführen von mehr als 2 Flüssen?

2.4 Erweiterte Konnektoren

- 2.4.1 Folgender Prozesskettenauschnitt aus dem Arbeitsfluss eines Großwildfängers ist gegeben. Wie muss die Entscheidungstabelle aussehen damit die Funktion sinnvoll ausgeführt werden kann?



- 2.4.2 Modifizieren Sie das Diagramm aus Aufgabe 2.4.1 mit Hilfe eines Kombinationskonnektors so, dass parallel zum Käfig füllen Futter beschafft wird.

Hausübung

2.5 OCL Anwendung IV (12 Punkte)

Betrachten Sie das von Übungszettel 1 (Abb. 1) bekannte UML-Klassendiagramm. Drücken Sie folgende Constraints in Form von OCL-Ausdrücken aus.

Verwenden Sie dabei ausschließlich die Operationen, die auf Übungsblatt 1 (in den Aufgaben 1.3 und 1.7) eingeführt wurden.

1. Bei allen Veranstaltungen sind Gruppentickets teurer als Einzeltickets!
2. Nachdem in einer Veranstaltungshalle eine Veranstaltung ausgerichtet wurde, werden von dem zuständigen Manager direkt 10 % der Ticketeinnahmen dem Budget der Veranstaltungshalle gutgeschrieben
3. Nach dem Wechsel zu einem neuen Arbeitgeber verdient ein Manager aus dem Bereich Personalwesen 10 % mehr als bei seinem alten Arbeitgeber, wenn er bei dem neuen Arbeitgeber weisungsbefugt ist in seiner neuen Abteilung mehr Mitarbeiter führen muss als in seiner alten. Zudem befindet er sich durch die Weisungsbefugnis in der nächsthöheren Gehaltsklasse
4. Die Veranstaltungshalle kann den Ticketverkauf einer Veranstaltung zu einer bestimmten Datum abfragen. Dabei wird die aktuelle Verkaufszahl zurück gegeben!
5. Um den Besuchern Abwechslung zu bieten, haben die Manager der Veranstaltungshallen beschlossen im Jahr 2015 nicht mehr als 40 Flohmärkte und 100 Rockkonzerte auszurichten!

2.6 Text zu Modell (8 Punkte)

2.6.1 Modellieren Sie eine zu dem Text passende EPK.

Der Prozess beschreibt das Reservieren von Kino-Tickets mit einer Smartphone-App.

Der Kunde möchte Karten reservieren. Dies geschieht online. Zunächst wird geprüft, ob ein Kundenkonto besteht. Sollte noch keines bestehen, wird es angelegt (die Kreditkarten-Daten werden hierbei u.a. mit abgefragt).

Im Anschluss kann der Kunde seinen Reservierungswunsch angeben. Die Kreditkarte des Kunden wird belastet und der Kunde bekommt einen QR-Code für die Ticket-Abholung übertragen.

Hat der Kunde 1 Stunde vor Vorstellungsbeginn seine Karten nicht abgeholt, indem er am Ticketautomaten den QR-Code scannt, wird der Ausdruck der Tickets verweigert.

Modellieren Sie den zeitlichen Aspekt mithilfe einer Entscheidungstabelle (ET).

Softwarekonstruktion – Übung 3

3 Petrinetze + Metamodellierung

3.1 Petrinetze und Metamodelle

Gegeben sei folgende Definition eines S/T -Netzes:

Ein S/T -Netz $ST = (S, T, F, K, W, M_0)$ ist ein 6-Tupel mit:

- (S, T, F) ist ein Petri-Netz, d.h.
 - S ist endliche, nicht-leere Menge von Stellen
 - T ist endliche, nicht-leere Menge von Transitionen
 - S und T sind disjunkt (d.h. $S \cap T = \emptyset$) und bilden die sog. *Knotenmenge* des S/T -Netzes
 - F ist endliche, nicht-leere Menge von Flüssen (Kanten) mit $F \subseteq (S \times T) \cup (T \times S)$
- $K : S \rightarrow \mathbb{N} \cup \{\infty\}$ ist Kapazitätsfunktion, die jeder Stelle eine Kapazität zuordnet.
Achtung: Bei der Bearbeitung der nachfolgenden Aufgabe sei zur Vereinfachung die default-Kapazität ∞ nicht zulässig!
- $W : F \rightarrow \mathbb{N}_+ \setminus \{0\}$ ist Gewichtsfunktion, die jeder Kante ein positives Gewicht zuordnet.
- $M_0 : S \rightarrow \mathbb{N}$ ist eine Markierungsfunktion (Anfangsmarkierung), die jeder Stelle eine Anzahl von Marken zuordnet, wobei gilt $\forall s \in S : M(s) \leq K(s)$.

Geben Sie ein Metamodell in UML für die Modellierung von S/T -Netzen an.

Modellieren Sie dabei folgenden Begriffe: S/T -Netz, Knoten, Stellen, Marken, Kapazität einer Stelle, Transitionen, Kanten, Vorbereich, Nachbereich, Quelle einer Kante, Senke einer Kante, Gewicht einer Kante (Kantenvielfachheit) **Achtung: in Aufgabe 3.2 sei zur Vereinfachung die default-Kapazität ∞ nicht zulässig!** Verwenden Sie die 5 Klassen Knoten, Transitionen, Stellen, Kanten und S/T -Netz! Achten Sie beim Modellieren darauf, dass alle Rollennamen eindeutig sind!

3.2 Metamodelle und OCL

Untersuchen Sie Ihr in Aufgabe 3.1 erstelltes Metamodell und stellen Sie fest, ob sich damit unzulässige Petrinetze modellieren lassen. Falls ja, an welcher Stelle?

Benutzen Sie OCL-Ausdrücke in dem Kontext *Kante*, um die Modellierungsschwäche zu beheben!

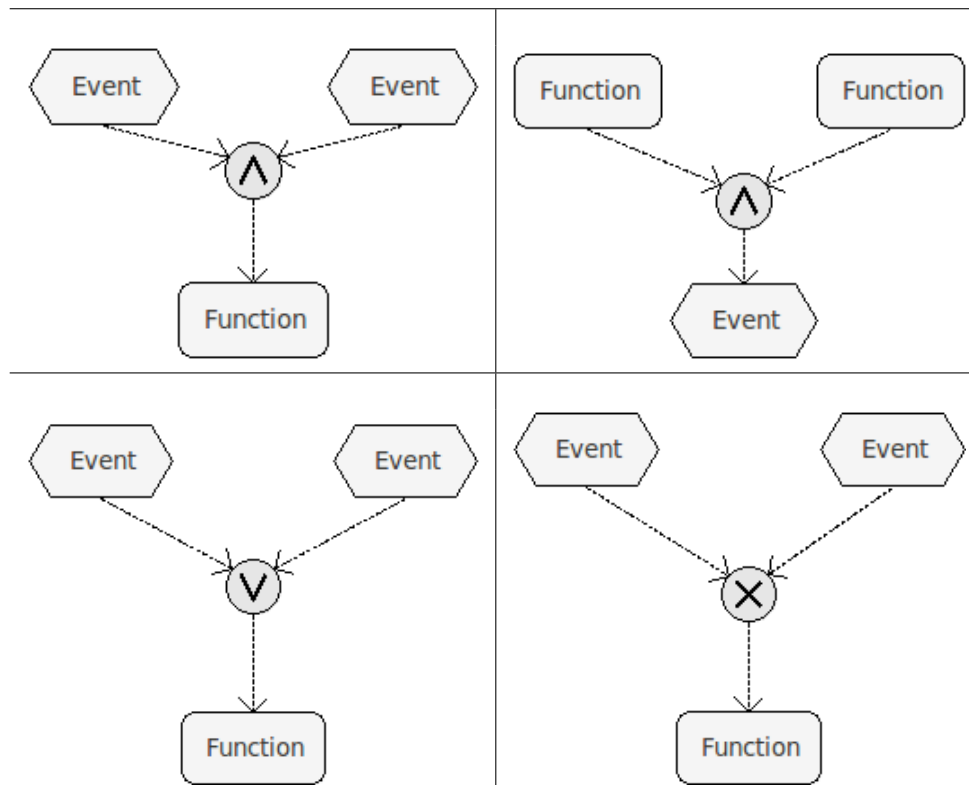
3.3 Petrinetze - Netzeigenschaften

Sei $ST = (S, T, F, W, K, M_0)$ ein S/T-Netz.

- Erklären Sie die formalen Definitionen aus den Vorlesungsfolien umgangssprachlich und verständlich.
 - Eine Transition $t \in T$ heißt **aktivierbar im gesamten Netz** $\Leftrightarrow \exists M_i \in [M_0 > \text{mit: } M_i[t >$
 - Eine Transition $t \in T$ heißt **lebendig im gesamten Netz** $\Leftrightarrow \forall M_i \in [M_0 > \text{gilt: } \exists M_j \in [M_i > \text{mit: } M_j[t >$
 - Eine Transition $t \in T$ heißt **tot im gesamten Netz** $\Leftrightarrow \forall M_i \in [M_0 > \text{gilt: } \neg M_i[t >$
- Erklären Sie den Zusammenhang zwischen einer lebendigen Transition und einem lebendigen S/T-Netz sowie zwischen einer toten Transition und einem toten S/T-Netz umgangssprachlich.

3.4 Petrinetze - Transformation

Transformieren Sie folgende EPK-Ausschnitte in gültige Petrinetze.

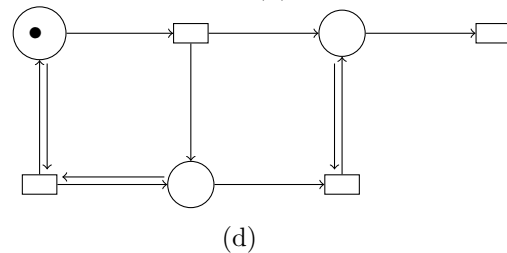
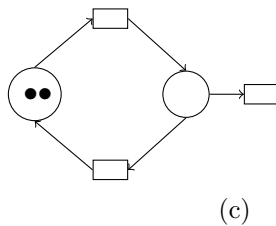
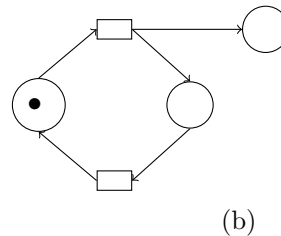
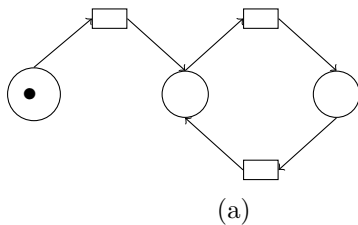


Hausübung

3.5 Petrinetze - Netzeigenschaften (5 Punkte)

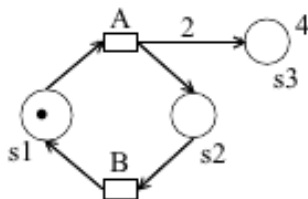
- Geben Sie für die nachstehenden Petrinetze (a) - (d) jeweils an, ob das **gesamte Netz** lebendig, deadlockfrei oder tot ist. Geben Sie zudem für jedes der Netze an, ob es beschränkt ist und nennen Sie im positiven Falle das kleinste n an für welches das Netz n -sicher ist.

Füllen Sie dazu untenstehende Tabelle aus.



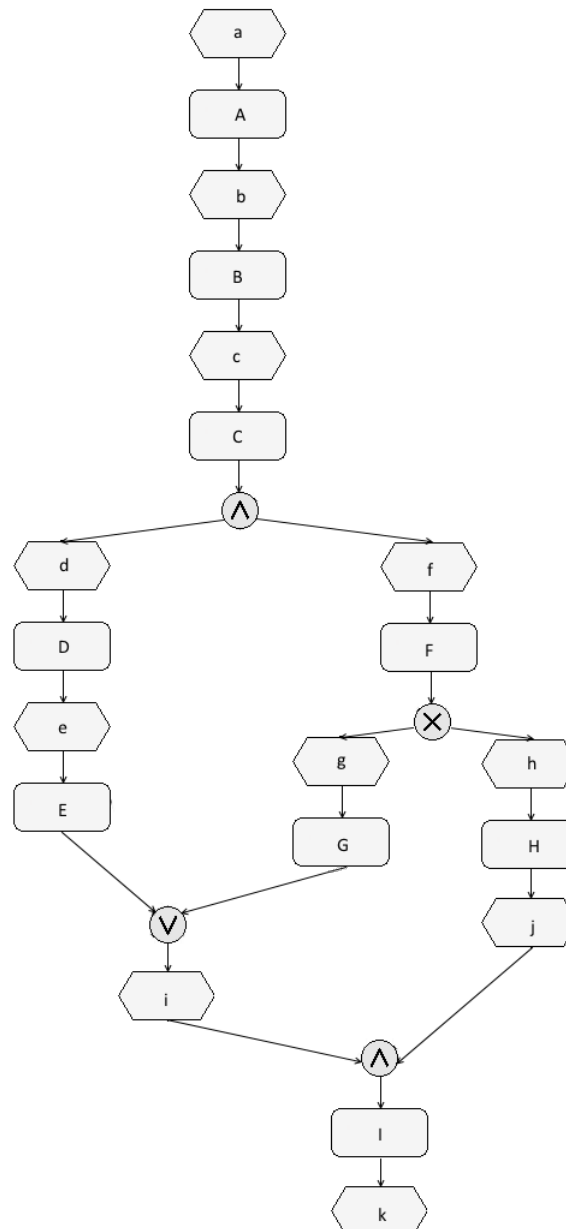
	a)	b)	c)	d)
lebendig				
deadlockfrei				
tot				
beschränkt				
n -sicher mit n				

- Geben Sie zu dem nachfolgenden Petrinetz den Erreichbarkeitsgraphen an. Zeichnen Sie den Erreichbarkeitsgraphen vollständig mit allen erreichbaren Markierungen M_i sowie den für die Zustandsübergänge verantwortlichen Transitionen A und B . Achten Sie auf Kantengewichte und Stellenkapazitäten.



3.6 Petrinetze - Transformation (5 Punkte)

Betrachten Sie die nachfolgend gegebene EPK.



1. Transformieren Sie die EPK **so wie sie gegeben ist** in ein S/T-Netz.
2. Untersuchen Sie ihr entstandenes Netz auf Deadlocks. Bei welcher Transition kann ein Deadlock auftreten?
3. Überlegen Sie, wie Sie das Petrinetz korrigieren könnten und skizzieren Sie Ihren Vorschlag stichpunktartig.

Softwarekonstruktion – Übung 4

4 Qualitätsmanagement und Metriken

4.1 Qualitätsmanagement

4.1.1 *Welche zwei grundlegenden Arten von Qualitätsstandards gibt es? Erklären Sie diese kurz.*

4.1.2 *Erläutern Sie den Unterschied zwischen verifizierenden, analysierenden und statisch bzw. dynamisch testenden Prüfverfahren. Nennen Sie jeweils ein Beispiel.*

4.2 Standards im Qualitätsmanagement

4.2.1 *Was versteht man unter ISO9001 und CMMI? Welche Ziele verfolgen sie?*

4.2.2 *Welchen der beiden Standards würden Sie nutzen für?*

1. Dokumentation, Überprüfbarkeit oder Transparenz von bestehenden Qualitätsprozessen
2. Bewertung oder Verbesserung des Qualitätsmanagements

4.3 Grundlagen Metriken

- 4.3.1 *Nennen Sie fünf statische Metriken und erläutern Sie diese kurz.*
- 4.3.2 *Setzen sie statischen Metriken aus der vorherigen Aufgabe in Bezug zu den Qualitätsmerkmalen Wartbarkeit, Verlässlichkeit, Wiederverwendbarkeit, Benutzbarkeit, Portabilität und Komplexität. Falls Sie einem der Qualitätsmerkmale keine Metrik zuordnen können, schlagen Sie eine weitere vor.*
- 4.3.3 *Was ist ein grundlegendes Problem bei der Verwendung von Metriken?*
- 4.3.4 *Erklären Sie warum es schwierig sein kann den Zusammenhang von internen Produkteigenschaften und externen Produkteigenschaften zu validieren. Nehmen Sie als Beispiel dafür Zyklomatische Komplexität und Wartbarkeit.*

4.4 Zyklomatische Komplexität

4.4.1 *Geben Sie an, wie sich Sequenzen, Verzweigungen und Sprünge auf die zyklomatische Komplexität auswirken.*

4.4.2 *Zeichnen Sie den Kontrollflussgraphen zu folgender Funktion und bestimmen Sie ihre zyklomatische Komplexität.*

```
void output( int selection , int y){
    if (selection ==1){
        printf("\n_1");
    }
    else {
        if (selection ==2) {
            printf("\n_2");
        }
        else{
            if (selection ==3){
                printf("\n_3");
            }
            else{
                if (selection ==4){
                    if (y == 0){
                        printf("\n_4");
                        printf("\n_5");
                    }
                    else{
                        printf("\n_6");
                    }
                }
                else{
                    printf("\n_7");
                }
            }
        }
    }
}
```

Hausübung

4.5 Anwendung von Metriken (5 Punkte)

- 4.5.1 *Geben Sie mindestens drei Gründe dafür an, die Ursache dafür sein könnten, dass Softwaremetriken in der Industrie oft keine Anwendung finden. (1 Punkt)*
- 4.5.2 *Nehmen Sie an, dass Sie für ein Unternehmen arbeiten, das Datenbanken für Einzelpersonen und kleine Unternehmen entwickelt. Das Unternehmen ist nun an der Messung des Erfolgs der Softwareentwicklung interessiert. Erarbeiten Sie einen Vorschlag der folgendes enthält: Metriken, wie diese Metriken gemessen und gesammelt werden können und eine Beschreibung des Zwecks, den jede einzelne Metrik hat. Achten Sie darauf Ihren Vorschlag auf das oben beschriebene Unternehmen und seine Produkte zuzuschneiden. Ihr Vorschlag sollte mindestens fünf Metriken enthalten. Begründen Sie Ihre Wahl. (4 Punkte)*

4.6 Zyklomatische Komplexität II (5 Punkte)

- 4.6.1 *Zeichnen Sie den Kontrollflussgraphen zu folgender Funktion und geben Sie ihre zyklomatische Komplexität an. (2 Punkte)*

```
int wandleDezimalZahl() {  
    int Dezimalzahl = 0;  
    int Potenzwert = 0;  
    char Zchn;  
    Zchn = leseNaechsteZeichen();  
    while (((Zchn == "0") || (Zchn == "1")) && (Dezimalzahl < INT.MAX)){  
        if (Zchn == "1") {  
            Dezimalzahl = Dezimalzahl + Math.pow (2, Potenzwert);  
        }  
        Potenzwert = Potenzwert + 1;  
        Zchn = leseNaechsteZeichen();  
    }  
    return Dezimalzahl;  
}
```

- 4.6.2 *Betrachten Sie erneut den Sourcecode des Programms aus Aufgabe 4.6.1. Analysieren Sie das Programm, indem Sie es folgenden statischen Metriken (lt. Vorlesung) unterziehen. (3 Punkte)*

- Fan-in / Fan-out
- NCSS

- NOC
- Länge der Bezeichner

Geben Sie dazu jeweils an:

- Ist es sinnvoll / möglich, diese Metrik für das angegebene Programm zu berechnen?
Wenn nein, warum nicht?
- Berechnen Sie die Metrik, falls möglich.
- Welche Aussage lässt der Zahlenwert zu (falls die Metrik zu berechnen war, mit Bezug auf das Programm)?

Softwarekonstruktion – Übung 5

5 Blackbox-Testen, Whitebox-Testen

5.1 Grundlagen Äquivalenzklassen und Grenzwert

Gegeben ist die Methode `berechneIBAN` mit folgender Signatur:

```
1 public String berechneIBAN(int Kontonummer){  
2     // Der Code der Methode ist unbekannt  
3 }
```

Die Methode `berechneIBAN` wird innerhalb einer Bank in Deutschland eingesetzt und berechnet die IBAN zu einer beliebigen Kontonummer der Bank; sie wird im Rückgabewert (String) zurückgegeben.

Der ermittelte String soll 22 Zeichen lang sein. Die ersten beiden Zeichen stehen für das Länderkennzeichen, in diesem Fall „DE“ für Deutschland. Anschließend folgt eine zweistellige Prüfziffer, die achtstellige Bankleitzahl und eine zehnstellige Kontonummer.

Hat die eingegebene Kontonummer weniger als zehn Stellen, so wird sie linksbündig mit Nullen auf zehn Stellen aufgefüllt.

Sollte es bei der Berechnung zu einem Fehler kommen, gibt die Methode `berechneIBAN` den String „Error“ zurück.

5.1.1 Welche Äquivalenzklassen von Eingabewerten sind sinnvollerweise zu testen?

5.1.2 Welche Sonder- und/oder Randfälle sind sinnvollerweise ebenfalls zu testen?

5.1.3 Geben Sie für jede Äquivalenzklasse, sowie für jeden Sonder- und/oder Randfall einen möglichen Eingabewert und das Soll-Ergebnis an. Bitte verwenden Sie hierzu keine realen Kontonummern. Benutzen sie stellvertretend für die Prüfsumme und Bankleitzahl die Strings „pp“ und „bbbbbbbb“

5.2 Kontrollflussbezogenes Testen – Testerfüllung

Das gegebene Programmsegment soll getestet werden:

```
1 read (x, y)
2 if ( x==0 ) { x:= 1 }
3 if ( (x>0) AND (y>0) ) { x:=x*y }
```

5.2.1 *Skizzieren Sie das Flussdiagramm.*

5.2.2 *Geben Sie alle Entscheidungswege an.*

Nehmen Sie für die folgenden Aufgaben an, es würde ein Test mit den Testdaten (x=0, y=1) und (x=-1, y=5) durchgeführt.

5.2.3 *Erfüllt diese Testmenge die Anweisungsüberdeckung? Begründen Sie Ihre Antwort.*

5.2.4 *Erfüllt diese Testmenge die Zweigüberdeckung? Begründen Sie Ihre Antwort.*

5.2.5 *Erfüllt diese Testmenge die Entscheidungsüberdeckung? Begründen Sie Ihre Antwort.*

5.2.6 *Erläutern Sie den Unterschied zwischen einer Entscheidungs- und einer Zweigüberdeckung. Was ändert sich, wenn man den Abbruch von Tests mit einbezieht?*

5.2.7 *Erfüllt diese Testmenge die einfache Bedingungsüberdeckung? Begründen Sie Ihre Antwort.*

5.3 Kontrollflussbezogenes Testen – Testfälle

Gegeben ist das folgende Programm. Es wandelt eine Binärzahl in eine Dezimalzahl um. Die Stellen der Binärzahl werden invers eingelesen, d. h. die letzte Ziffer zuerst, danach die vorletzte usw.

```
1 #define INT_MAX 200
2 int wandleDezimalZahl(){
3     int zahl = 0;
4     int potenzwert = 0;
5     char zeichen;
6     scanf("%c", &zeichen);
7     while ( ( ( zeichen == "0" ) || ( zeichen == "1" ) )
8             && ( zahl < INT_MAX ) ) {
9         if ( zeichen == "1" ) {
10            zahl = zahl + pow(2, potenzwert);
11        }
12        potenzwert++;
13        scanf("%c", &zeichen);
14    }
15    return zahl;
16 }
```

5.3.1 Zeichnen Sie den zum Programm gehörenden Kontrollflussgraphen.

5.3.2 Erstellen Sie Testfälle für einen minimalen, aber vollständigen Anweisungsüberdeckungstest (C0). Geben Sie den durchlaufenen Pfad an.

5.3.3 Erstellen Sie Testfälle für einen minimalen, aber vollständigen Zweigüberdeckungstest (C1). Geben Sie den durchlaufenen Pfad an.

Hausübung

Die Hausübung findet online statt!

Alle nötigen Informationen (beachten Sie insbesondere die Abgabefrist) sind der Webseite zur Vorlesung zu entnehmen:

<https://www-secse.cs.tu-dortmund.de/secse/pages/teaching/ws14-15/swk/>

bzw.

https://www-secse.cs.tu-dortmund.de/secse/pages/teaching/ws14-15/swk/index_de.shtml#onlineuebungen

Softwarekonstruktion – Übung 6

6 Whitebox-Testen II

6.1 Grenze-Inneres-Überdeckung

Das gegebene Programmsegment soll mit einem Kontrollfluss-Testverfahren getestet werden:

```
1 while ( (a<b) OR (c<d) ) {  
2     a:= a + 1;  
3     c:= c + 1;  
4 }
```

1. Geben Sie einen minimalen Satz von Testdaten an, der für die Bedingung der while-Schleife die minimal bestimmende Mehrfachbedingungsüberdeckung erfüllt.
2. Geben Sie einen minimalen Satz von Testdaten an, der die Grenze-Inneres-Überdeckung erfüllt.

Gegeben sei nun ein Programmsegment und sein aus einem Refactoring entstandenes Gegenstück.

1 factorial= counter;	1 /* Refactoring */}
2 counter--;	2 factorial=1;
3 while (counter > 0) {	3 do {
4 factorial= factorial * counter;	4 factorial= factorial * counter;
5 counter--;	5 counter--;
6 }	6 } while (counter > 0)
7 System.out.println(factorial);	7 System.out.println(factorial);

3. Geben Sie jeweils Testfälle für eine Grenze-Inneres-Überdeckung an. Worin unterscheiden sich die beiden Testfälle? Wodurch kommt der Unterschied zu Stande?

Auch das folgende Programmsegment soll mit einem Kontrollfluss-Testverfahren getestet werden:

```
1  x=a; y=b;
2
3  /* ggT (klassisch): */
4  while (a!= b) {
5      if (a>b) { a= a-b; }
6      else {b= b-a; }
7  }
8
9  /* ggT (modern): */
10 tmp = 0;
11 while (y != 0) {
12     tmp = x % y;
13     x = y;
14     y = tmp;
15 }
```

4. Geben Sie für den Testdatensatz $\{(8, 2), (5, 5), (6, 3)\}$ den Grenze-Inneres-Überdeckungsgrad an.
5. Geben Sie einen minimalen Satz von Testdaten an, der die Grenze-Inneres-Überdeckung erfüllt.

6.2 Kontrollflussbezogenes Testen – Testfälle II

Das gegebene Programmsegment soll mit einem Kontrollfluss-Testverfahren getestet werden:

```
1 read ( x , y , z )
2 if ( ( x > 0 ) AND ( y > 0 ) AND ( z > 0 ) ) { x := x * y * z }
3 else { x := y }
4 if ( x + y + z > 0 ) { z := -x }
```

6.2.1 Geben Sie eine minimale Testmenge für eine Anweisungsüberdeckung an.

6.2.2 Geben Sie eine minimale Testmenge für eine einfache Bedingungsabdeckung an.

6.2.3 Geben Sie eine minimale Testmenge für eine Mehrfachbedingungsabdeckung an.

6.2.4 Geben Sie eine minimale Testmenge für eine minimal bestimmende Mehrfachbedingungsabdeckung an.

6.2.5 Geben Sie eine minimale Testmenge für eine Pfadabdeckung an.

6.3 Kontrollflussbezogenes – Refactoring

Bei einem Refactoring wurde das Programmsegment aus Aufgabe 6.2 wie folgt geändert:

```
1 read(x,y,z)
2 if ( x>0 ) {
3     if ( y>0 ) {
4         if ( z>0 ) { x:=x*y*z }
5         else { x:=y }
6     }
7     else { x:=y }
8 }
9 else { x:=y }
10 if ( x+y+z>0 ) { z:=-x }
```

Das Programmsegment soll nun erneut getestet werden:

- 6.3.1 *Geben Sie eine minimale Testmenge für eine Anweisungsüberdeckung an. Vergleichen Sie Ihr Ergebnis mit dem Ergebnis von Aufgabe 6.2.1 und begründen Sie gegebenenfalls den Unterschied.*
- 6.3.2 *Geben Sie eine minimale Testmenge für eine einfache Bedingungsabdeckung an. Vergleichen Sie Ihr Ergebnis mit dem Ergebnis von Aufgabe 6.2.2 und begründen Sie gegebenenfalls den Unterschied.*
- 6.3.3 *Geben Sie eine minimale Testmenge für eine Mehrfachbedingungsabdeckung an. Vergleichen Sie Ihr Ergebnis mit dem Ergebnis von Aufgabe 6.2.3 und begründen Sie gegebenenfalls den Unterschied.*
- 6.3.4 *Geben Sie eine minimale Testmenge für eine minimal bestimmende Mehrfachbedingungsabdeckung an. Vergleichen Sie Ihr Ergebnis mit dem Ergebnis von Aufgabe 6.2.4 und begründen Sie gegebenenfalls den Unterschied.*

Hausübung

Die Hausübung findet online statt!

Alle nötigen Informationen (beachten Sie insbesondere die Abgabefrist) sind der Webseite zur Vorlesung zu entnehmen:

<https://www-secse.cs.tu-dortmund.de/secse/pages/teaching/ws14-15/swk/>

bzw.

https://www-secse.cs.tu-dortmund.de/secse/pages/teaching/ws14-15/swk/index_de.shtml#onlineuebungen